

TD VAT 1 : Analyse de pire durée d'exécution

La durée d'exécution d'un programme dépend de plusieurs facteurs :

- les instructions élémentaires le composant ;
- le processeur et sa vitesse. Naïvement, on pourrait penser que chaque instruction prend 1 cycle, et donc, fréquence 2x plus élevée donne traitement 2x plus rapide. Ceci s'approche de la réalité sur des microcontrôleurs ancienne génération, mais est de plus en plus faux. C'est en tout cas totalement faux sur un microprocesseur ;
- l'état interne du processeur, qui peut par exemple contenir des pipelines ;
- l'état de la mémoire cache : l'instruction à exécuter, et les données requises, sont-elles en cache (accès 1 à 2 cycles) ou dans la RAM (accès en 3 à 10 cycles) ou stockées sur le disque dur (swap) car nous utilisons de la mémoire virtuelle (accès en plusieurs dizaines de milliers de cycles)

Il existe deux familles de méthodes pour analyser le WCET (Worst-Case Exécution Time) d'un programme sur un calculateur.

- Analyse statique de WCET, utilise un programme qui doit avoir un modèle précis du calculateur employé. Exemples : OTAWA, aiT, RapiTime, Heptane, etc. Sûre (jamais optimiste) mais nécessite un modèle précis de l'architecture d'exécution. Pessimiste lorsqu'il y a des caches et pipelines car toute inconnue sur l'état du système est modélisée de façon conservative.
- Analyse dynamique de WCET (voir Figure 1), qui consiste à instrumenter le code avec des mesures d'horloge (logicielles ou matérielles – horloges dans le CPU, analyseur logique), et mesurer de nombreuses exécutions. La difficulté est qu'il est généralement impossible de mesurer tout état interne et tout chemin (couverture du code). La solution est donc non sûre.
 - Solutions mixtes avec considération de la couverture ;
 - WCET probabiliste – distribution cumulative (cdf) de pourcentage des exécution vs. pire durée.

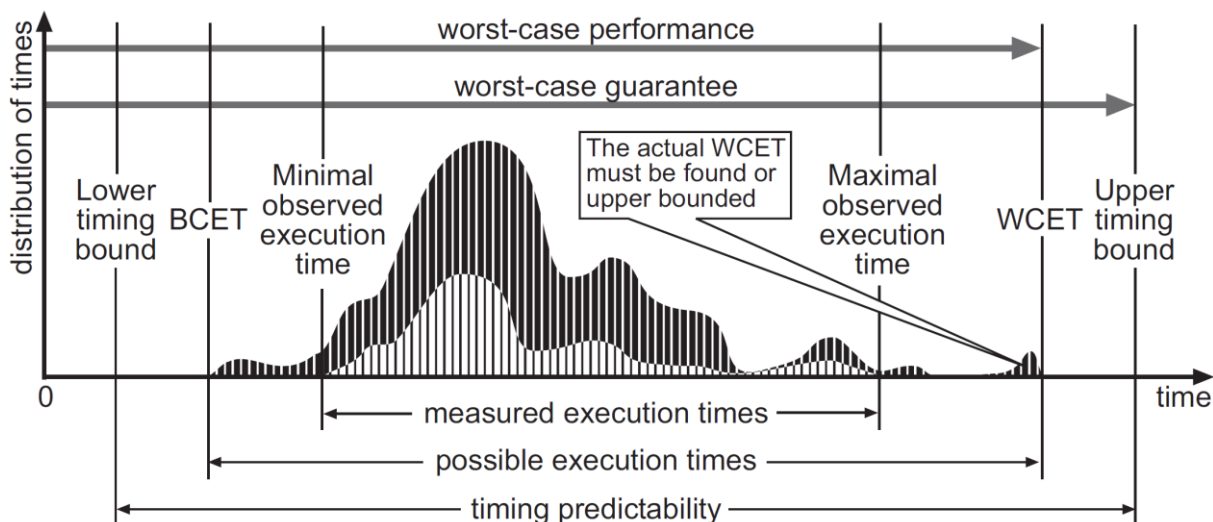


Figure 1 : Distribution typique d'une durée d'exécution, selon [1], l'un des survey les plus cités dans le domaine de la Timing Analysis (analyse de WCET)

1 Analyse statique avec OTAWA

Note : la partie du TD portant sur OTAWA a été conçue par Antoine Bertout (LIAS).

Nous allons tester OTAWA qui est un framework d'analyse statique développé à l'IRIT (Institut de Recherche en Informatique de Toulouse). Dans son principe, OTAWA décode le binaire du programme, construit son CFG (graphe de flot de contrôle) et abstrait finement le matériel. Il combine ces informations pour borner le nombre de fois que les différentes parties du programme (blocs de bases) peuvent être exécutés et le temps (nb cycles) d'exécution de chacune de ces parties. Nous ferons l'hypothèse que ce programme est destiné à être exécuté sur un microcontrôleur ARM LPC2138 de la compagnie NXP car le modèle de cette architecture est déjà disponible dans OTAWA.

1.1 Préparation

Sous Linux :

```
xhost +local:

docker run -it --rm -e DISPLAY=:0.0 --net=host registry-
ia.ensma.fr/svtr /bin/bash
```

Sous Windows avec WSL2 :

```
docker run -it --rm -v /tmp/.X11-unix:/tmp/.X11-unix -e DISPLAY -e
WAYLAND_DISPLAY -e XDG_RUNTIME_DIR -e PULSE_SERVER --net=host
registry-ia.ensma.fr/svtr /bin/bash
```

1.2 Utilisation d'OTAWA

Plusieurs programmes C sont disponibles dans le répertoire wcet, comme bs.c avec une simple fonction main qui appelle la fonction de recherche dichotomique binary_search dans un tableau.

1. Commencer par visualiser le programme.

```
cd wcet ; cat bs.c
```

2. Compiler le programme avec gcc pour architecture ARM (on parle de compilation croisée)

```
arm-linux-gnueabi-gcc -g3 -fno-stack-protector -marm -march=armv4 -o
bs.elf bs.c -nostartfiles -static -T link-flash.ld;
```

3. Lancer l'estimation de WCET de la fonction main avec OTAWA.

```
owcet -s lpc2138 bs.elf
```

En essayant d'interpréter la sortie de owcet, on peut comprendre qu'OTAWA a besoin de connaître le nombre maximal de fois que la boucle de la fonction binary_search peut être appelée (borne de boucle). Quelle est cette borne N (pour rappel, le tableau possède 15 éléments) ?

4. Nous allons préciser cette borne manuellement en éditant (avec vim) un fichier ff (flowfact) produit par la commande suivante.

```
mkff bs.elf > bs.ff
```

Dans bs.ff remplacer le symbole ? par la valeur de N trouvée précédemment. Dans vim, taper i pour passer en mode édition, échap pour sortir du mode édition, et :wq pour sauvegarder et quitter vim.

5. Relancer owcet en liant le fichier de flow fact et obtenir le WCET du programme sur le microcontrôleur.

```
owcet -s lpc2138 bs.elf -f bs.ff
```

6. Fort heureusement, l'outil orange d'OTAWA permet de déterminer automatiquement cette borne de boucle, vérifiez la valeur trouvée dans le fichier ffx en comparant avec la vôtre.

```
orange bs.c main -o bs.ffx
```

7. Nous allons générer un ensemble de statistiques qui vont nous permettre de visualiser le CFG du programme annoté.

```
owcet -s lpc2138 bs.elf --stats -S
```

8. Lancer l'interface graphique de visualisation du CFG.

```
obviews.py bs main
```

9. Recompilez le programme avec l'option -O1, relancez la commande owcet du 7. Et observez les différences de WCET.

2 Analyse dynamique

Télécharger bs.c sur votre machine, à partir du dossier Ressources du TD sur Moodle. La fonction de recherche dichotomique est la même que celle que nous avons mesurée statiquement pour la cible LPC2138. Supposons que le programme doive s'exécuter sur le CPU de votre machine. Il est probable que le modèle de votre CPU ne soit pas disponible pour un analyseur de WCET statique. Nous allons donc effectuer des mesures dynamiques.

1. Compiler et lancer plusieurs fois le programme. Comment peut-on expliquer les variations observées ?

2. Modifier la valeur d'indice de fin de boucle for, en la passant à 10, 100, 1000. Comment expliquez-vous la variation décroissante de la moyenne observée de chaque durée d'exécution ?

3 Bibliographie

- [1] R. Wilhelm *et al.*, « The worst-case execution-time problem—overview of methods and survey of tools », *ACM Trans. Embed. Comput. Syst.*, vol. 7, n° 3, p. 36:1-36:53, mai 2008, doi: 10.1145/1347375.1347389.